06 — Entry 37: trạng thái dang dở (để chạy tiếp)

Viết 2026-11-03, branch `hmm-navigation-5`. Phiên bị dừng giữa chừng theo yêu cầu user. File này ghi toàn bộ việc đã làm, trạng thái, quyết định thiết kế, số liệu thiết kế đã thấy và các bước còn lại, đủ để một phiên mới chạy tiếp mà không cần lịch sử chat. Đọc kèm: `paper/BAO_CAO_E36.pdf`, `RESEARCH_LOG.md` Entry 36, `paper/cms_theory.md` §4f.

00. Cập nhật phiên 2026-11-03 (đang chạy; đọc trước)

- Token 7 acc đã cài (`~/.kaggle/tok_<acc>`), `automation/active_accounts.txt` = thứ tự §2. Đăng ký trước Entry 37 đã commit (`RESEARCH_LOG.md` Entry 37) TRƯỚC mọi đánh giá.
- **Kernel thăm dò `nhatlong1103/e37p-1`** (L4, 62 job thật của đợt 1, 46 song song): mọi job rc = 0, 17 phút. Dưới tải đầy: ms/frame (GPU cho phép thử tuyến) CMU 34–47, Nordland 89, office 96, old town 87 (numpy 158), countryside 83, Boreas 83–87 (numpy 153), NCLT 103, Oxford 113; GPU nhanh hơn numpy 1.3× (CMU) – 1.8× (Boreas); **GPU và numpy trùng tuyệt đối** (3/3 file); bản gộp từ kernel trùng bản numpy cục bộ (chỉ `tau` lệch 8e-9, BLAS). RAM đỉnh ≤ 3.1 GB/job, GPU ≤ 0.23 GB/job.
- **Đánh giá gộp vào đợt 1** (`analysis/e37_eval.py`): mỗi run là một đơn vị đóng gói (mọi phần + job đánh giá trong cùng kernel); job đánh giá chờ `.done` của các phần, gộp (`results/fwd/fwd_*.npz`), chạy `oc_curves --fams @E36,@E37 --cms-groups cond,loco --cst --navgate --gate-rate --dump-frames` → `oc_*_e37_s*.json` + `frames_*_e37_s*.npz`. Bỏ được một vòng đẩy/kéo.
- **Đợt 1 = 13 kernel L4 `e37w-1..13`** (`automation/e37_wave.py automation/e37_frames.json automation/e37_costs.json <groups> 13 --skip`, 2 146 job: 1 527 phần, 300 run đánh giá, 45 chuẩn bị Oxford, 274 N = 40; ~27 phút/kernel ước tính) trên 7 acc theo thứ tự (2/acc; `caotienminh1` 1). 12 run CMU seed 0–1 đã có null tiến từ e37p-1 → đánh giá cục bộ (xong).
- **St Lucia:** frame (10 m, 6 lượt) đã đăng `nhatlong1103/stlucia-vpr-frames`; kernel L4 `stl37-1` (`automation/make_stlucia_kernel.py`: 6 descriptor + patch DINOv2 ViT-L/14 từ model Kaggle `metaresearch/dinov2/pyTorch/large/1` + chuỗi E37 cho 6 × seed 0–4 + SafeVPR'26 holes/no-hole); bản thử `stlt-1` (400 frame/lượt) đạt mọi bước.
- Theo dõi: một `central_monitor.py` (state ở scratchpad); nếu container mất: kéo đầu ra bằng `KAGGLE_API_TOKEN=$(cat ~/.kaggle/tok_<acc>) kaggle kernels output <acc>/e37w-<k> -p <dir>` (acc theo thứ tự đẩy trong §2).
- Code mới: `analysis/e37_eval.py`, `analysis/e37_audit.py` (H37b), `analysis/e37_safevpr.py`, `analysis/envrun.py`, `paper/build_e37_report.py`, `automation/make_stlucia_kernel.py`, `automation/e37_costs.json`; `paper/cms_theory.md` §4g (Mệnh đề 19); `LITERATURE_GATE.md` Gate 16.

0. Việc được giao (user, sau vòng 10)

1. Phẫu thuật tiến cả **bằng chứng hiện diện**, để CST-J thành bảo đảm thật.
2. Tìm **null chính xác cho đầu chuỗi** (vấn đề CMU).
3. Khi có GPU: Boreas đủ 18 run, N = 40 cho mọi descriptor, St Lucia với 6 descriptor.

Quy định user nhắc lại trong phiên này: (a) tối ưu triệt để — dồn hết GPU/CPU/VRAM của 1 acc rồi mới sang acc khác, thời gian chạy tối thiểu; (b) **code xong phải tự rà lại code + công thức (đúng chưa, tối ưu chưa) và chạy thử đầu-cuối trước khi chạy thật**, tránh lỗi giữa chừng.

1. Trạng thái tóm tắt

Hạng mục	Trạng thái
Code phẫu thuật tiến cho bằng chứng hiện diện (`cms_fwd.FwdStats`)	**Xong, đã kiểm**
Khởi đầu ảo đủ chân trời (mode 2) + giữ khởi đầu 3 excl cũ (mode 1) trong cùng một lượt	**Xong, đã kiểm**
Backend GPU torch float64 cho phép thử tuyến (`analysis/cms_torch.py`)	**Xong, kiểm trên torch-CPU** (chưa đo tốc độ trên GPU thật)
Luật E37 (CST-JS và các ablation) trong `cms.py` / `oc_curves.py` (`@E37`)	**Viết xong, chạy được trên 2 run thiết kế**; chưa đăng ký trước, chưa đánh giá

Hạng mục	Trạng thái
Chia một run theo chuỗi (`--part k/n`, `--merge`), job kernel (`e37_fwd.py`, `e37_prep.py`, `n40_run.py --one`)	**Xong, smoke test runner Kaggle giả lập đạt (4/4 job rc = 0)**
Gói đợt 1 vào 14 kernel L4 (`automation/e37_wave.py`)	**Xong** (file nhóm sinh lại được), **CHƯA đẩy lên Kaggle**
Đăng ký trước Entry 37 trong `RESEARCH_LOG.md`	**CHƯA viết** (phải viết trước khi xem kết quả đánh giá)
St Lucia 6 descriptor (việc 3c)	**Chưa bắt đầu** (cần kernel GPU có internet)
Báo cáo / handoff 00–05 / theory §4g	**Chưa**

Không có kernel Kaggle nào đang chạy do phiên này tạo. Không có tiến trình nền nào cần giữ.

2. Tài khoản Kaggle (user cấp 2026-11-02)

7 acc: `nhatlong1103, quynhle07, quynhtran35, caotienminh1, nguyenhoa99, yuan1103, synchan5704`. Token **không** nằm trong git; phiên mới phải xin lại user, lưu vào `~/.kaggle/tokens_raw` (chmod 600) và `~/.kaggle/tok_<acc>` (mỗi file chỉ chứa token, chmod 600), `~/.kaggle/access_token` = token của `nhatlong1103`. Kiểm: `KAGGLE_API_TOKEN=$(cat ~/.kaggle/tok_<acc>) kaggle config view`. Lúc kiểm (2026-11-02): cả 7 token hợp lệ, không bị giới hạn API. `caotienminh1` có kernel `zc-probe` của phiên khác — **không đụng**. Thứ tự dồn slot đề xuất (L4 trước): `nhatlong1103, quynhle07, nguyenhoa99, synchan5704, yuan1103, quynhtran35, caotienminh1` → ghi vào `automation/active_accounts.txt` (hiện file vẫn là danh sách cũ, **chưa sửa**).

3. Phương pháp mới (E37) — đã hiện thực

3a. Phẫu thuật tiến của bằng chứng hiện diện (`analysis/cms_fwd.py`)

- `FwdStats`: thống kê hiện diện **của chính hệ thống** (chi phí tuyến C, winner j, độ khác biệt z — đúng phép tính của `cms_exact` / `cms_vec`) tính trên **bản đồ đã phẫu thuật của từng lần chạy ảo**: giả thuyết = top-K của frame trên bản đồ còn lại (từ hàng similarity đầy đủ), bảng tra cứu của mọi frame trong cửa sổ cũng trên bản đồ đó (kể cả các frame trước lối thoát), vệt anchor rời chunk khi đi vào vùng bị xoá (`HoleMask`: cắt Voronoi như lỗ trên bản đồ; khởi đầu: hành lang ρ). Khởi đầu ảo: cửa sổ cắt tại điểm bắt đầu.
- `fwd_null(..., presence=True, start_h=(None, "full"))`: lưu thêm `costf, zf, jf` cho mỗi lag; mode 0 = lối thoát, mode 1 = khởi đầu 3 excl (đúng E36), mode 2 = khởi đầu đủ chân trời off_d + 20 excl, hố = hành lang ρ quanh toàn bộ đường tin. `fs.run_stats` tính theo lô lớn cho cả một lần chạy ảo (thống kê hiện diện chỉ phụ thuộc bản đồ của lần chạy, không phụ thuộc chuỗi).
- `cms_fwd.py --check-presence <cms.npz>`: trên bản đồ nguyên vẹn, **trùng cache công bố**: cost 100 %, z 100 %, j 99.83 % (hoà fp16, như vòng trước 99.85 %), CMU CosPlace s3.
- `--part k/n` (chia chuỗi cân bằng theo số frame, `part_seqs`), `--merge out parts...`.

3b. Backend GPU (`analysis/cms_torch.py`)

`cms_vec._route_cost` tự chuyển sang torch float64 khi thấy GPU (`CMS_DEVICE=cuda|cpu|numpy`, mặc định auto). Cùng phép tính từng bước (nội suy đúng thuật toán `np.interp`, `torch.searchsorted` thay cho `_searchsorted_rows`, cùng offset tra cứu, `CorridorMask` / `HoleMask` có `.torch`). **Kiểm (torch-CPU vs numpy): 100 % trùng, sai khác 0** trên null tiến (costf, zf, jf, rr, gidx; 20 286 lag) và trên thống kê phản thực đầy đủ của `cms_vec` (8/8 đầu ra). Chưa đo tốc độ GPU thật — đo ở kernel đầu tiên (xem §6 bước 2).

3c. Luật E37 (`analysis/cms.py`, khối `if len(recs[0]) > 9 ...`; preset `oc_curves.py --fams @E37`)

- `cms.chain_xform`: biến đổi thống kê → log-LR **của chính hệ thống** (hạng trong pool phản thực của nhóm, Beta-EM, lũy thừa e-value), dùng để chuyển thống kê tiến thành log-LR.
- `cms_fwd.joint_crit`: mô phỏng quyết định vector hoá trên các lần chạy ảo (chuỗi hiện diện tiếp tục từ trạng thái thật ở lối thoát; khởi đầu ở odds 1), trả (exit, frames, vị trí, giá trị tới hạn dung sai, **log-odds lớn nhất của ứng viên cửa sổ ngắn**, cờ cửa sổ ngắn).
- `cms_fwd.eps_start_pos`: ngưỡng null khởi đầu **theo vị trí kể từ đầu chuỗi** (vị trí = chỉ số frame // H, biết không cần nhãn); bin ≥ 2/(a/2) đoạn; bin có cửa sổ ngắn chia a/2 (dung sai) + a/2 (ngưỡng log-odds κ′ cho frame cửa sổ ngắn).
- Họ luật:
- **`CMS-CPC/H+CSTJS+G(+NGRr)`** (ứng viên luật chính E37): lối thoát = ngưỡng min theo bin vị trí (bằng chứng hiện diện tiến); frame trong chân trời đầu chuỗi phải qua thêm ngưỡng khởi đầu theo vị trí; frame cửa sổ ngắn (< 1 excl) qua iff log-odds > κ′.

- `CMS-CPC/H+SH+CSTJS+G(+NGRr)`, `[both]` (nhân quả).
- Ablation: `CSTJS0` (cửa sổ ngắn không chứng nhận), `CSTJSL` (cấu trúc CSTJS nhưng hiện diện = phẫu thuật đầy đủ E36), `CSTJF0` (khởi đầu min theo bin, không cửa sổ ngắn).
- Kiểm hồi quy: mọi luật E36 (CST-F, CST-J, CST-J*, SH, SH ∧ CST, …) tính lại từ null tiến mới **trùng tuyệt đối** kết quả công bố E36 (CMU CosPlace s3 và NetVLAD s3, file `conf.tgz`).

3d. Phát hiện thiết kế (nhãn chỉ để đánh giá; CMU seed 3, các run giữ ra của E36 nơi CST-J* thất bại — từ nay là dữ liệu THIẾT KẾ)

- **Mép chuyến:** tỷ lệ đoạn ảo vị trí 0 mà hệ thống báo: **0.224** với hiện diện tiến (chính xác) so với **0.045** với hiện diện phẫu thuật đầy đủ (E36). ⇒ CST-J của E36 đánh giá thấp rủi ro ở mép gấp ~5 lần — giải thích cơ chế CST-J vượt α trên CMU giữ ra. Ngưỡng dung sai lối thoát ở mép (CMU CosPlace): 14.6 m (SH chọn tay 12.5 m).
- **Đầu chuỗi:** ở vị trí 0–2 (≈ 25 m đầu, cửa sổ < 1 excl) mô phỏng chính xác cho thấy hệ thống báo sai trên cửa sổ ngắn ở > 5 % khởi đầu ảo (11.6 % có ít nhất một báo cửa sổ ngắn) ⇒ "cho cửa sổ ngắn qua" của CST-J* không chứng nhận được; ngưỡng $\kappa' = 5.76$ (log-odds) thì hợp lệ và 77 % frame cửa sổ ngắn đúng trên bản đồ vượt κ'.
- Số liệu điểm mặc định (recall / segFAR), CMU s3:

Luật	CosPlace	NetVLAD
SH (CMS-CPC/H+SH+G+NGRr)	0.855 / 0.011	0.500 / 0.000
CST-J* (E36, thăm dò)	0.868 / 0.011	0.568 / 0.014
CST-F (E36)	0.678 / 0.000	0.348 / 0.000
CST-JS (E37)	**0.826 / 0.000**	**0.526 / 0.011**
SH ∧ CST-JS	0.826 / 0.000	0.475 / 0.000
CST-JS0 (không cửa sổ ngắn)	0.685 / 0.000	0.426 / 0.011
CST-JSL (hiện diện E36)	0.863 / 0.005	0.554 / 0.014
CST-JS [both]	0.812 / 0.000	0.487 / 0.008

Đọc: CST-JS giữ phần lớn recall của CST-J* (−0.04) và cao hơn CST-F nhiều (+0.15 / +0.18), với bảo đảm từ mô phỏng chính xác. Chỉ 2 run thiết kế — **chưa kết luận gì**.

4. Tốc độ đo được (1 lõi CPU, numpy)

- Cấu hình E36 (không hiện diện): 3.4–8 ms/frame. Cấu hình E37 (hiện diện tiến + 2 họ khởi đầu): CMU 19 ms/frame, Nordland 69, 4S office 91, old town 102, Oxford ~90 ms/frame (~45 lần chạy ảo sống mỗi frame × ~2 ms phép thử tuyến). ~70 % là `_route_cost` ⇒ lý do viết backend GPU. Phần còn lại (chuỗi vị trí) chạy CPU.
- Chuẩn bị Oxford (`e37_prep.py --no-fwd`): navcache 32 s, top-300 2 s, thống kê phản thực 115 s.

5. Đợt 1 trên Kaggle (đã dựng, CHƯA đẩy)

`python3 automation/e37_wave.py automation/e37_frames.json <groups.txt> 14` → 1 896 job vào 14 kernel (134–137 job/kernel, chênh tải < 0.3 %): 1 575 phần null tiến (main: seed 0–2 × 6 dataset có lỗ × 6 descriptor + NCLT s0 × 6; giữ ra: seed 6–9 × 6 × 6; thiết kế: CMU s3–5 × CosPlace/NetVLAD/MegaLoc; Oxford: seed 0–4 × 6 + seed 5–9 × SALAD/MixVPR/DINOv2, mỗi run Oxford = 1 job chuẩn bị + 7 phần chờ `--wait`, cùng một kernel), 276 job N = 40 (việc 3b: seed 1–9 × 6 dataset × 5 descriptor ≠ CosPlace + Boreas CosPlace s4–9). Boreas đủ 18 run (việc 3a) nằm trong tập main (null tiến chứa cả mode 1 ⇒ luật E36 tính được từ cùng file).

Dựng + đẩy (sau khi có token, sửa `active_accounts.txt` như §2):

```
python3 automation/fanout.py <work> e37w1 <groups.txt> --groups --l4 --par 46 --build-only --ds nhatlong1103/
```

```
python3 automation/central_monitor.py <work>/state.json --gap 60 --run-poll 600 --backoff 1800 --nocopy <work
```

(`--par 46`: 1 luồng BLAS/job trên 48 CPU của phiên L4; GPU chia vòng theo chỉ số job.) Smoke test runner: đã chạy `run.py` thật trên `/kaggle` giả lập (thư mục thật + symlink file — `os.walk` không theo symlink thư

mục) với 4 loại job: cả 4 rc = 0, 184 s. **Rủi ro:** nếu L4 kẹt, `central_monitor` tự chuyển sang 2×T4 (4 CPU) sau 5 lần — với 135 job/kernel sẽ rất lâu: theo dõi, nếu xảy ra thì xoá và chia nhỏ kernel đó. Đầu ra mỗi kernel: `results.tar` gồm `fwdp_*.npz`, `navcache/cms/topk/tao` của Oxford, `cms40/*`, `oc_*_n40_s*.json`, `oc_*_n20_s*.json`.

6. Các bước tiếp theo (thứ tự)

1. **Viết đăng ký trước Entry 37 vào `RESEARCH_LOG.md` TRƯỚC khi đánh giá** (đề xuất): luật chính CMS-CPC/H+CSTJS+G+NGRr; tham chiếu SH, SH ∧ CST, CST-F, CST-J*; ablation CSTJS0, CSTJSL, CSTJF0; nhân quả [both]; α = 0.05, nhóm cond + loco. Giả thuyết gợi ý: H37a segFAR TB ≤ 0.05 trên 7/7 + Oxford **và trên tập giữ ra seed 6-9 kể cả CMU** (nơi CST-J* vỡ: 0.068); H37b kiểm toán theo vị trí (mép / sâu, kiểu thoát / kiểu đầu chuỗi): tỷ lệ đoạn off-map thật có báo ≤ 0.07; H37c recall CST-JS ≥ CST-F + 0.05 trên CMU và trong 0.03 của SH trên ≥ 4/7; H37d sai số điều hướng trong 1 m của SH trên ≥ 5/7; H37e Oxford hợp lệ, recall so với SH 0.729. Ghi rõ: CMU s3 (CosPlace/NetVLAD) là run thiết kế; số liệu §3d đã thấy.
2. Có token → sửa `active_accounts.txt`, đẩy **một** kernel đầu (acc 1) và xem log 5–10 phút đầu: ms/frame với GPU (in ở cuối mỗi job `e37_fwd`), RAM, GPU memory; nếu ổn đẩy 13 kernel còn lại (central_monitor tự dồn: 2 GPU/acc theo thứ tự).
3. Kéo kết quả, gộp phần: `cms_fwd.py --merge fwd_<tag>_<desc>_s<seed>.npz fwdp_<...>.p*of*.npz`. Đăng dataset public `nhatlong1103/hmm-fwd-null-e37` (null tiến) và `nhatlong1103/hmm-cms-cache-n40` (`automation/publish_dataset.py`).
4. Đợt 2 (CPU, nhẹ): `oc_curves.py ... --cst --fwd <dir> --fams @E36,@E37 --cms-groups cond,loco --navgate --gate-rate` cho mọi run (lệnh đầy đủ như trong `analysis/e36_run.py` bước 3; Oxford thêm `--mapdump`). Dùng phiên CPU + L4 của các acc (dồn theo thứ tự). Kiểm toán theo vị trí: mở rộng `cst_audit.py` cho luật CST-JS (tỷ lệ đoạn off-map thật có báo, theo vị trí / kiểu chuyến).
5. N = 40 (việc 3b): đọc `oc_*_n40/n20` (preset @E32S), gộp với 48 run CosPlace của E36 (`analysis/results/e36/n40.tgz`), kiểm H36f trên mọi descriptor.
6. St Lucia 6 descriptor (việc 3c): kernel GPU **có internet** (T4, không phải L4): tải video từ HF `medwa126/UQStLuciaMultipleToD`, `automation/make_stlucia.py` (mỗi 5 m, rồi lấy mỗi 10 m như E36), trích 6 descriptor (trọng số `nhatlong1103/vpr-descriptor-weights`, `bundle`; mẫu: `automation/make_dump_boreas.py`, `extract_desc_cpu.py`), đăng `nhatlong1103/stlucia-vpr-dump`; rồi `e37_prep.py` cho stlucia (seed 0–4 × 6) + luật; SafeVPR'26 với DINOv2 **ViT-L/14** như bài báo (tải `facebook/dinov2-large` khi có internet).
7. Báo cáo `paper/BAO_CAO_E37.md/.pdf`, `paper/cms_theory.md` §4g (Mệnh đề cho CST-JS: lối thoát + khởi đầu theo vị trí + κ′, cận hợp a/2 + a/2), cập nhật handoff 00–05, ENVIRONMENT_SUMMARY; Gate 14 (novelty mô phỏng tiến).

7. Dữ liệu cục bộ (KHÔNG còn sau khi container bị thu hồi)

Scratchpad phiên cũ có: cache công khai đã tải, dump CMU/Nordland/4S/Oxford, null tiến E37 của CMU s3 CosPlace/NetVLAD (design), file nhóm đợt 1. Tất cả tái tạo được: `automation/anon_download.py` (navcache-v2, cms-cache-exact, route-cache, topk, vpr-descriptor-dumps `--match "(cmu|nord|meta|README)"`, vpr-dumps-4seasons, oxford-robotcar-vpr-dump — Oxford không có tiền tố `dump/`, cần symlink vào `dump/<desc>/`), rồi `analysis/e37_fwd.py <dump> <navcache_dir> cmu cosplace 3 0/1` (~7 phút/run CMU trên 1 lõi). Cài: `pip install kaggle scipy scikit-learn markdown xhtml2pdf` (+ `torch` CPU nếu muốn kiểm backend: `CMS_DEVICE=cpu`).

8. File thay đổi trong phiên (đã commit cùng file này)

`analysis/cms_fwd.py` (FwdStats, HoleMask, run_stats, mode 1/2, part/merge, joint_crit, eps_start_pos, check_presence), `analysis/cms_torch.py` (mới), `analysis/cms_vec.py` (dispatch GPU, CorridorMask), `analysis/cms.py` (chain_xform, luật CST-JS…), `analysis/oc_curves.py` (`_fwdsh`, @E37, danh sách [both] / cổng), `analysis/n40_run.py` (`--one`), `analysis/e37_fwd.py`, `analysis/e37_prep.py` (mới), `automation/e37_wave.py`, `automation/e37_frames.json` (mới).